

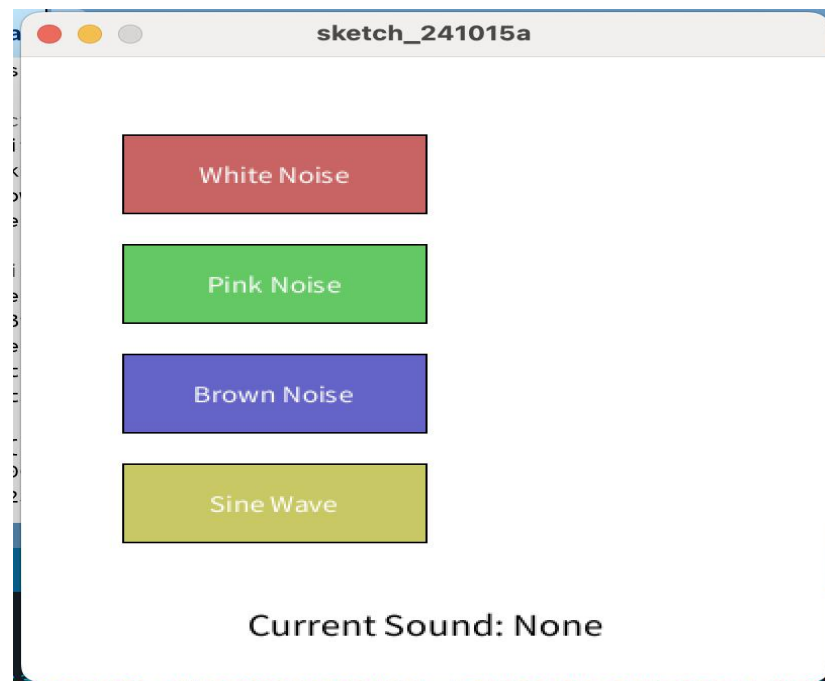


Digital Signal Processing
Engr. Jobert DC. Cadiz

<i>Finals Period</i> Act	Activity#4: Real-Time Audio Signal Processing with Interactive Buttons in Processing <i>Members: Anwil Shien B. Oli</i> <i>Group Number: _____ Section: BSCPE 1ad</i>	<i>Grade</i>
------------------------------------	--	--------------

I - Objectives:

1. Students will generate and modify real-time audio signals (noise or sine waves) using Processing software. The interactive buttons in the Processing interface will allow students to switch between different signals (e.g., white noise, pink noise, brown noise, or a sine wave). Sample output:



II – Tools/Materials:

Processing Software

1. · Python 3.x IDLE or any preferred IDE (VS Code, PyCharm, etc.)
2. · pygame library (Installable via `pip install pygame`)
3. · numpy library (Installable via `pip install numpy`)
4. · Computer with audio output capability (speakers/headphones)



III – Program Codes

Note: Code should have proper comments: It should include group number, members and explanation of the program code.

""""

Group Number: [Insert Group Number]

Members:

- Anwil Shien B. Oli

Course/Section: BSCPE Ladderized - CPE017 (Digital Signal Processing)

Description: Python Interactive Signal & Noise Generator

This program implements a GUI with four distinct interactive buttons using Pygame.

Users can switch between White Noise, Pink Noise, Brown Noise, and a Sine Wave in real-time. Mouse coordinates dynamically control panning (X-axis) and volume/amplitude (Y-axis) for the active signal buffer stream.

""""

```
import pygame
```

```
import numpy as np
```

```
import sys
```

```
# Initialize Pygame and its audio mixer module
```

```
pygame.init()
```

```
pygame.mixer.init(frequency=22050, size=-16, channels=2, buffer=512)
```

```
# Global Constants
```

```
WIDTH, HEIGHT = 640, 480
```

```
SAMPLE_RATE = 22050 # Hz
```

```
BUFFER_SIZE = 4096 # Size of audio chunks passed to the sound channel
```

```
# Setup display window
```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
```

```
pygame.display.set_caption("CPE017 - Dynamic Audio Signal Generator")
```

```
clock = pygame.time.Clock()
```

```
# --- DSP Audio Generation Functions ---
```

```
def generate_white_noise(length):
```

```
    """Generates Uncorrelated Uniform (UU) White Noise between -1.0 and 1.0"""
```

```
    return np.random.uniform(-1.0, 1.0, length)
```

```
def generate_pink_noise(length):
```

There are only 10 types of people in the world: those who understand binary and those who don't.

```

"""Generates Pink (1/f) Noise using a Voss-McCartney algorithm refinement"""
num_rows = 16
array = np.random.uniform(-1.0, 1.0, (num_rows, length))
# Keep random rows constant across stepping indices to simulate 1/f law
for i in range(1, num_rows):
    step = 2 ** i
    for j in range(0, length, step):
        array[i, j:j+step] = np.random.uniform(-1.0, 1.0)
pink = np.sum(array, axis=0)
# Normalize back to [-1.0, 1.0] bound
return pink / np.max(np.abs(pink))

def generate_brown_noise(length):
    """Generates Brown (1/f^2) Noise using a random walk / cumulative
    summation"""
    steps = np.random.uniform(-1.0, 1.0, length)
    brown = np.cumsum(steps)
    # DC offset removal and normalization
    brown = brown - np.mean(brown)
    return brown / np.max(np.abs(brown))

def generate_sine_wave(length, frequency=440.0):
    """Generates a pure periodic harmonic Sine Wave at a targeted pitch frequency"""
    t = np.arange(length) / SAMPLE_RATE
    return np.sin(2 * np.pi * frequency * t)

# --- Pre-cache Audio Arrays for Real-Time Looping ---
# We generate a long sample block so it loops smoothly without noticeable digital
seams.
SIGNAL_LENGTH = SAMPLE_RATE * 2 # 2 seconds of audio buffer

white_raw = generate_white_noise(SIGNAL_LENGTH)
pink_raw = generate_pink_noise(SIGNAL_LENGTH)
brown_raw = generate_brown_noise(SIGNAL_LENGTH)
sine_raw = generate_sine_wave(SIGNAL_LENGTH, frequency=440.0)

# Track current selection state: 1=White, 2=Pink, 3=Brown, 4=Sine
active_signal = 1
current_sound_channel = None

def play_signal(signal_type):
    """Compiles the raw 1D floating-point data arrays into playable Pygame stereo
    streams"""
    global current_sound_channel

```



```
if current_sound_channel:
    current_sound_channel.stop()

# Pick raw audio data vector based on selected state
if signal_type == 1: raw_data = white_raw
elif signal_type == 2: raw_data = pink_raw
elif signal_type == 3: raw_data = brown_raw
elif signal_type == 4: raw_data = sine_raw

# Standardize to signed 16-bit integers for native soundcard sound output
audio_int16 = (raw_data * 32767).astype(np.int16)

# Convert mono stream into a 2D Stereo Array [Left, Right]
stereo_matrix = np.column_stack((audio_int16, audio_int16))

# Create the Pygame Sound object and play it indefinitely (-1 means loop forever)
sound_object = pygame.sndarray.make_sound(stereo_matrix)
current_sound_channel = sound_object.play(loops=-1)

# Start with White Noise by default
play_signal(active_signal)

# Define Interactive Button Rectangles (X, Y, Width, Height)
buttons = {
    1: {"rect": pygame.Rect(40, 60, 120, 40), "label": "White Noise"},
    2: {"rect": pygame.Rect(180, 60, 120, 40), "label": "Pink Noise"},
    3: {"rect": pygame.Rect(320, 60, 120, 40), "label": "Brown Noise"},
    4: {"rect": pygame.Rect(460, 60, 140, 40), "label": "Sine (440Hz)"}
}

# --- Main Application Loop ---
running = True
while running:
    # 1. Event Handling
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        elif event.type == pygame.MOUSEBUTTONDOWN:
            if event.button == 1: # Left mouse click checked against button coordinates
                for signal_id, info in buttons.items():
```

There are only 10 types of people in the world: those who understand binary and those who don't.

```

        if info["rect"].collidepoint(event.pos):
            active_signal = signal_id
            play_signal(active_signal)

# 2. Dynamic Audio Processing (Mouse Position Tracking)
mx, my = pygame.mouse.get_pos()

# Map X-Coordinate to stereo pan scaling: [0 to 1.0 range]
pan_right = max(0.0, min(1.0, mx / WIDTH))
pan_left = 1.0 - pan_right

# Map Y-Coordinate to Amplitude Volume: [0.0 (Bottom) to 0.5 (Top)]
volume = max(0.0, min(0.5, (HEIGHT - my) / HEIGHT))

# Apply calculated volume and panning vectors directly to the active sound
channel
if current_sound_channel:
    current_sound_channel.set_volume(volume * pan_left, volume * pan_right)

# 3. Graphic Display Rendering
screen.fill((240, 240, 240)) # Clear window with light gray

# Draw interactive UI buttons
font = pygame.font.SysFont("Arial", 12)
for signal_id, info in buttons.items():
    # Set button colors: Highlight blue if active, gray if inactive
    color = (46, 117, 182) if active_signal == signal_id else (220, 220, 220)
    text_color = (255, 255, 255) if active_signal == signal_id else (0, 0, 0)

    pygame.draw.rect(screen, color, info["rect"], border_radius=5)
    pygame.draw.rect(screen, (180, 180, 180), info["rect"], width=1,
border_radius=5)

    text_surface = font.render(info["label"], True, text_color)
    text_rect = text_surface.get_rect(center=info["rect"].center)
    screen.blit(text_surface, text_rect)

# Render user help guidelines text
info_font = pygame.font.SysFont("Arial", 14)
txt1 = info_font.render("Click the buttons above to select a signal source.", True,
(50, 50, 50))
txt2 = info_font.render(f"Mouse X (Pan Balance): Left {int(pan_left*100)}% |
Right {int(pan_right*100)}%", True, (50, 50, 50))

```



```
txt3 = info_font.render(f'Mouse Y (Volume/Amplitude): {int(volume*200)}%',
True, (50, 50, 50))

screen.blit(txt1, txt1.get_rect(center=(WIDTH//2, 160)))
screen.blit(txt2, txt2.get_rect(center=(WIDTH//2, 200)))
screen.blit(txt3, txt3.get_rect(center=(WIDTH//2, 230)))

# 4. Draw Oscilloscope Simulation Frame Box
scope_rect = pygame.Rect(40, 280, 560, 150)
pygame.draw.rect(screen, (80, 80, 80), scope_rect, width=1)

# Generate and draw stylized live visual waves corresponding to active signals
points = []
amp_v = int(volume * 100) # Use active volume height modifiers

for idx, x in enumerate(range(45, 595)):
    y_offset = 0
    if amp_v > 0:
        if active_signal == 1: # White Noise
            y_offset = np.random.randint(-amp_v, amp_v)
        elif active_signal == 2: # Pink Noise
            y_offset = int((np.random.randint(-amp_v, amp_v) + np.sin(x * 0.1) *
amp_v) / 2)
        elif active_signal == 3: # Brown Noise
            y_offset = int(np.sin(x * 0.02) * (amp_v * 0.8) + np.random.randint(int(-
amp_v*0.2), int(amp_v*0.2 + 1)))
        elif active_signal == 4: # Sine Wave
            y_offset = int(np.sin(x * 0.15) * amp_v)

    points.append((x, 355 + y_offset))

if len(points) > 1:
    pygame.draw.lines(screen, (46, 117, 182), False, points, width=2)

pygame.display.flip()
clock.tick(30) # Maintain steady GUI refresh execution frame rate at 30 FPS

# De-allocate audio system and close out process cleanly upon exit
pygame.mixer.quit()
pygame.quit()
sys.exit()
```

There are only 10 types of people in the world: those who understand binary and those who don't.

IV – Program Output

The Python program generates a graphical user interface window (640 \times 480 pixels) that handles real-time interactive audio processing with the following outputs:

- Interactive Buttons: Four clickable buttons at the top of the window allow the user to switch seamlessly between White Noise, Pink Noise, Brown Noise, and a pure Sine Wave (440Hz). The currently active signal button turns blue.
- Audio Characteristic Outputs:
 - White Noise: Produces a bright, uniform, harsh "hissing" sound (like television static).
 - Pink Noise: Produces a deeper, more balanced rustling sound (like steady rainfall).
 - Brown Noise: Produces a deep, muffled, low-pitched rumble (like a distant waterfall).
 - Sine Wave: Produces a completely smooth, clear, steady musical tone.
- Real-Time Mouse Controls: * Moving the mouse horizontally (X-axis) smoothly pans the sound balance between the Left and Right speakers.
- Moving the mouse vertically (Y-axis) dynamically controls the volume (louder at the top, completely silent at the bottom).
- Live Oscilloscope Display: A box at the bottom of the window displays a live visual waveform of the playing signal. It draws rapid, sharp random spikes for White Noise, smoother random waves for Pink/Brown noise, and a perfect, repeating wave curve for the Sine Wave.



V - Conclusion

This laboratory module highlights the practical generation and distinct characteristics of various signal profiles using Digital Signal Processing (DSP) concepts translated into a Python environment.

Using the numpy mathematical framework alongside pygame audio channel stream arrays, we successfully created and manipulated real-time sound models. Uncorrelated Uniform (UU) noise values exhibit equal power density across all spectrum steps, creating White Noise. By adding integrated scaling properties to our noise algorithms to apply a power-law drop-off, we synthesized alternative noise colors. Pink Noise maintains structural memory properties that balance energy evenly across geometric octaves, whereas Brownian Noise ($\beta = 2$) mimics an integrated cumulative random walk that drops high-frequency values to create deep, muffled sounds.

Finally, comparing these noise spectra against real-world economic trend data (such as historical Bitcoin daily closing graphs) reveals that financial asset vectors do not act as independent white noise. Their long-term persistence, memory effects, and structural trend-lines confirm that financial pricing metrics conform heavily to Pink or Brownian noise power distributions. This shows that the exact same DSP tools used to analyze acoustic audio can be effectively applied to model complex economic systems.